EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 10
Aim: Mini Project
Name: Mannat A. Nandi, Prem L. Panchal, Vallari R. Panvalkar, Ashish R. Patel
Roll Number: 51
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty: Dr. Yogesh Pingle

Aim: Mini Project

Abstract:

In today's fast-paced digital environment, individuals frequently struggle to maintain personal relationships due to high cognitive loads and schedule constraints, often resulting in missed milestones such as birthdays and anniversaries. Existing digital calendars and reminder applications exacerbate this issue by relying on passive, generic push notifications that contribute to user notification fatigue. To address this gap, this project introduces **KinSync**, a proactive, AI-powered relationship management system designed to completely automate personal milestone tracking and acknowledgment.

Developed using the MERN stack (MongoDB, Express.js, React.js, Node.js), KinSync operates as a secure, multi-tenant web application featuring JSON Web Token (JWT) authentication and an intuitive, responsive frontend dashboard. The system architecture innovatively integrates the Google Gemini API to dynamically generate highly personalized, context-aware greeting messages based on specific relationship dynamics. Furthermore, to guarantee immediate and reliable delivery while bypassing strict text-based API limitations, the platform utilizes the Bland AI API to autonomously dispatch real-time, personalized voice calls to recipients on the exact date of their milestone.

Experimental validation of the deployed application confirms the successful execution of secure user routing, real-time data summarization, AI-driven content generation, and automated telephonic dispatch. Ultimately, KinSync successfully shifts the burden of memory and execution from the user to an autonomous system, demonstrating a novel, zero-touch approach to preserving social connectivity and reducing cognitive load.

Objectives:

- To design a dynamic web application using front-end and back-end technologies
- To implement CRUD operations using PHP and MySQL
- To retrieve data from a public API using AJAX / Fetch API
- To implement dynamic search and filtering
- To ensure secure data handling using validation, sessions, and cookies
- To understand real-world client–server interaction

Problem Statement:

In today's fast-paced, digital-first world, maintaining strong personal and professional relationships requires consistent effort. However, people increasingly face high cognitive loads and busy schedules, making it difficult to remember and acknowledge personal milestones such as birthdays and anniversaries. This unintentional oversight frequently

leads to missed celebrations and weakened social ties, undermining the essential human need for social engagement.

While existing digital calendars and reminder applications attempt to address this, they predominantly offer generic, passive notifications. These traditional systems lack optimized prioritization, intuitive countdowns, and reliable communication automation. Consequently, these tools suffer from reduced relevance and poor user engagement regarding personal lifecycle reminders. Furthermore, dispatching basic push notifications at arbitrary times contributes directly to user stress and notification fatigue rather than solving the underlying issue.

Therefore, there is a critical need to design a proactive system that bridges the gap between busy daily schedules and social connectivity. To effectively preserve interpersonal bonds, a solution must be context-aware, timely, and personally meaningful leveraging real-time date tracking and AI-driven automation to shift the burden of memory and execution from the user to the system, ensuring significant milestones are never missed.

Proposed system:

➢Introduction

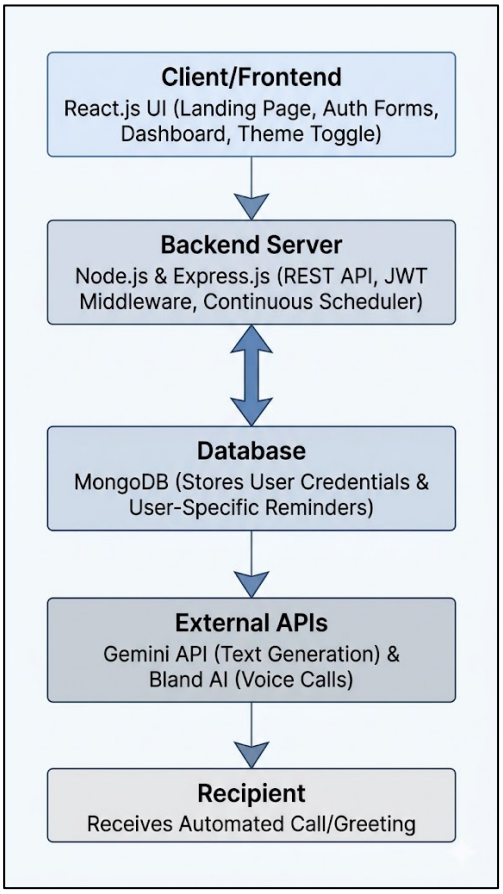
To overcome the limitations of passive digital calendars and the rising issue of user notification fatigue, the proposed system, KinSync, introduces a proactive, multi-tenant automation approach to relationship management. Driven by the critical need to reduce the cognitive load associated with modern, busy schedules, the system shifts the burden of memory and execution entirely from the user to a secure, autonomous architecture.

Unlike earlier iterations or standard reminder tools that rely on generic push alerts, this advanced system leverages live artificial intelligence to handle communication. It integrates the Gemini API to dynamically generate context-aware, highly personalized greeting messages, and utilizes Bland AI to initiate automated voice calls to the recipient.

Furthermore, the system has been upgraded into a fully secure SaaS platform. By implementing robust user authentication via JSON Web Tokens (JWT) and bcrypt password hashing, KinSync provides a secure, isolated environment for individual users to manage their data. The frontend interface has also been significantly enhanced with a dedicated landing page, secure login routing, and dynamic theme toggling (light/dark mode), ensuring a highly accessible and organized user experience.

➢Architecture / Framework / Block Diagram

System Architecture Overview KinSync is built on a modern, secure Client-Server architecture utilizing the MERN stack (MongoDB, Express.js, React.js, Node.js). It is specifically designed to handle multi-tenant data securely while orchestrating third-party AI and communication APIs.



Block Diagram Flow (Guide for your visual diagram)

Framework Components

1. Presentation Layer (React.js Frontend) The client side is developed as a Single Page Application (SPA). It now features conditional routing that directs unauthenticated users to a Landing Page and Authentication portal (`Auth.js`), and authenticated users to their private dashboard. The interface captures user inputs and securely manages session states using tokens stored in local storage. Additionally, it includes state-driven theme toggling (Light/Dark mode) to improve accessibility. All data exchanges with the server are handled asynchronously via Axios, attaching Authorization headers to secure endpoints.

2. Application Layer (Node.js & Express.js Backend) The server acts as the central orchestrator. It handles secure user registration and login by comparing hashed passwords and issuing JSON Web Tokens (JWT). A custom authentication middleware protects core routes, ensuring users can only read, create, or delete their own specific reminder records. The backend also hosts the core scheduling engine, which continuously monitors the database for matching dates and triggers the appropriate execution functions.

3. Data Storage Layer (MongoDB) The NoSQL database has been restructured to support a relational multi-tenant model. It utilizes two primary Mongoose schemas: a `User` schema (storing names, emails, and bcrypt-hashed passwords) and a `Reminder`

schema. The `Reminder` schema includes a `userId` foreign key that directly links every contact and milestone to the specific user who created it, ensuring total data isolation and privacy.

4. Integration Layer (External AI APIs) To deliver on the promise of zero-touch, personalized automation, the backend integrates with two distinct third-party services. When an event is triggered or requested by the user, the server queries the **Gemini API** to generate a unique, context-aware greeting based on the recipient's name and occasion. For delivery, the system interfaces with **Bland AI**, pushing the finalized payload to initiate an automated, real-time voice call to the recipient, completely bypassing standard text notification fatigue.

➢Technologies Used

Client-Side Scripting:

Language / Library	Purpose
JavaScript (React.js)	UI components and client-side logic
CSS3 (Custom Properties)	Responsive styling and dark/light theming
Axios	HTTP requests to the backend REST API

Server-Side Scripting:

Language / Library	Purpose
JavaScript (Node.js + Express)	REST API server and route handling
MongoDB + Mongoose	Database and schema modelling
JWT + bcryptjs	Authentication and password hashing
Google Gemini AI	AI-generated personalised messages
Bland AI	Automated AI voice call delivery
node-cron	Daily scheduler for auto-sending calls

➢Layout Design Snapshots

Screen	Layout Summary
Landing Page	Navbar · Hero with badge & CTAs · 6-card Features grid · 4-step How It Works · CTA banner · Footer
Auth Page	Animated background orbs · Centred card · Tab switcher (Log In / Sign Up) · Form fields · Submit button
Dashboard	Header · Left panel: Add Reminder form + Reminder cards grid · Right sidebar: Today's events + Coming Up

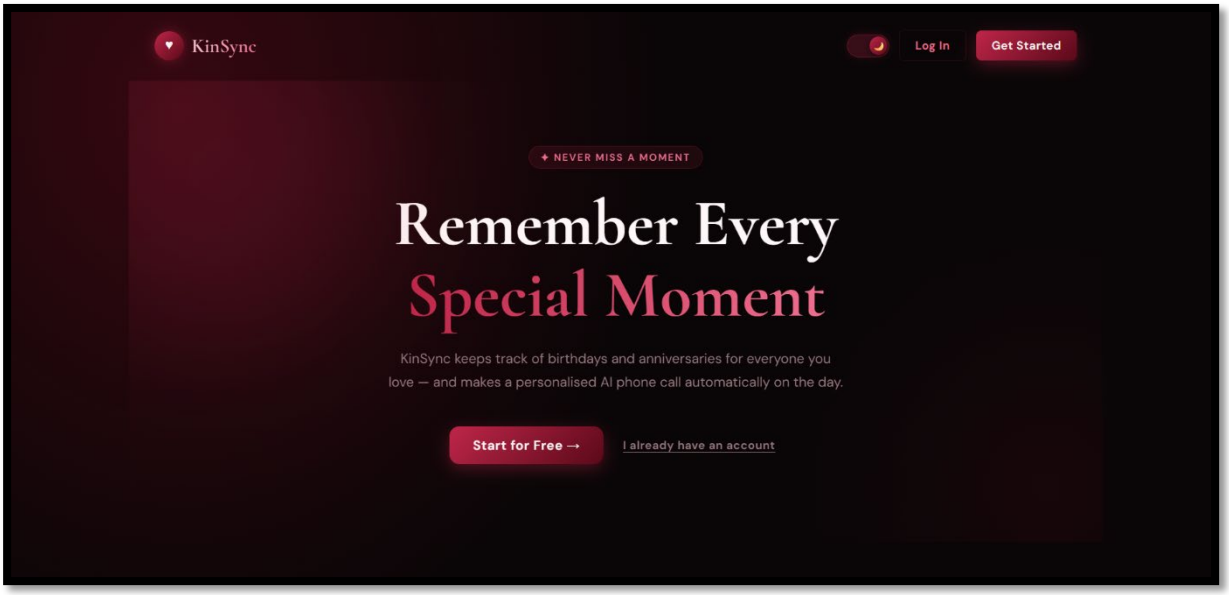


Figure 6: Landing Page (Dark theme)

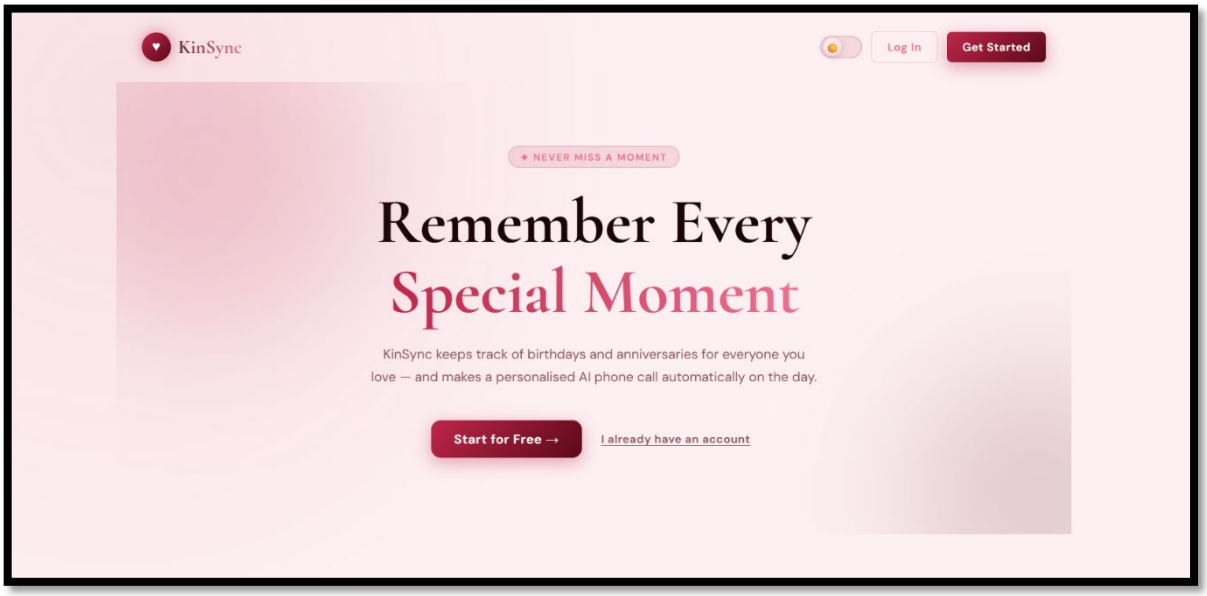


Figure 7: Landing Page (Light theme)

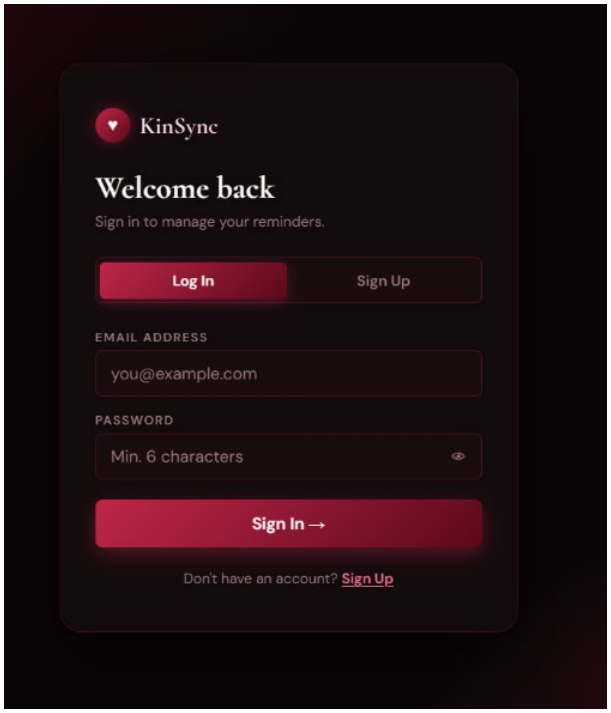


Figure 8: Log In (Dark Theme)

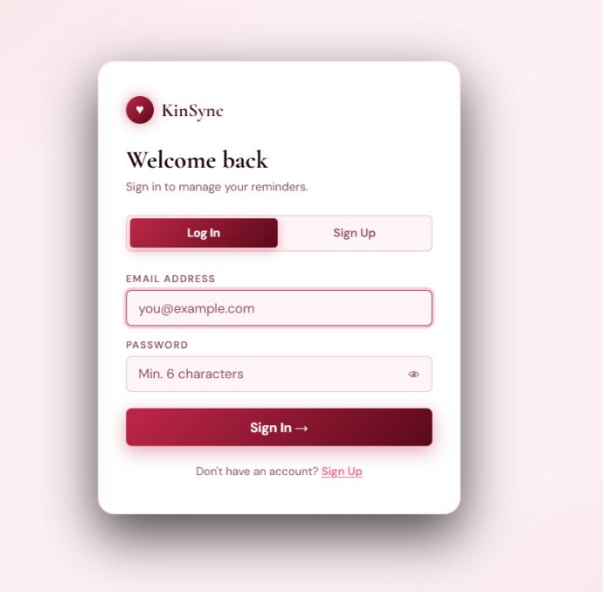


Figure 9: Log In (Light Theme)

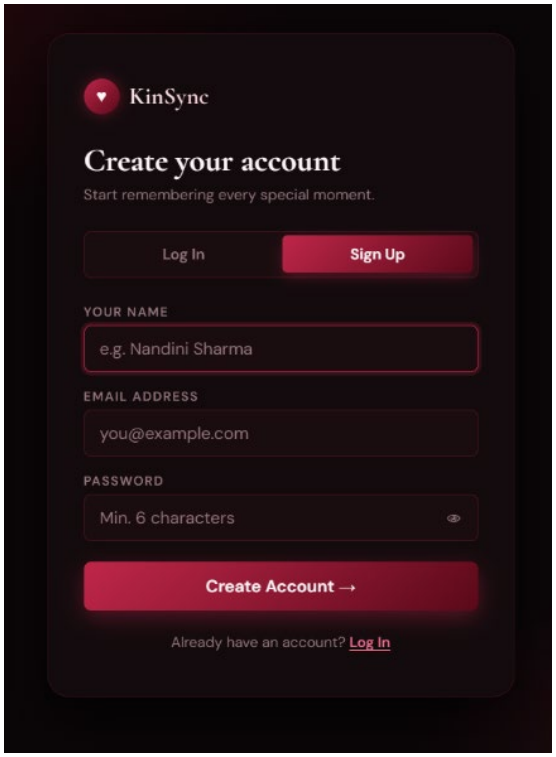


Figure 10: Sign Up (Dark Theme)

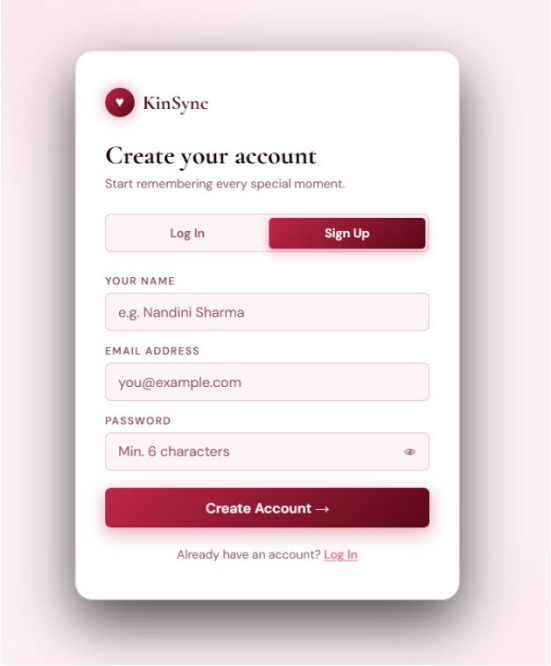


Figure 11: Sign Up (Light Theme)

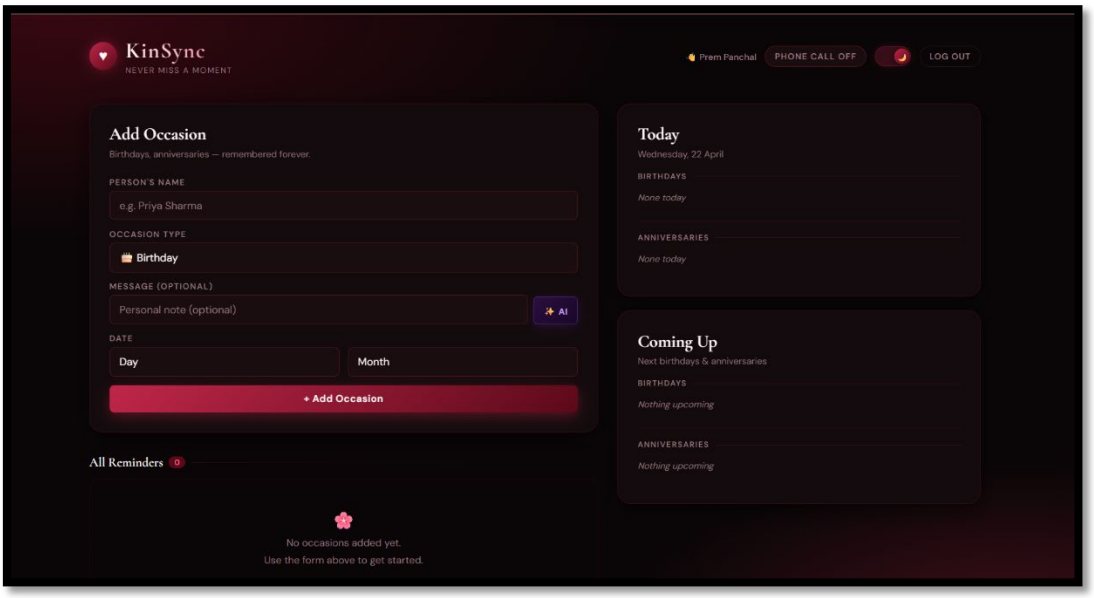


Figure 12: Dashboard (Dark Theme)

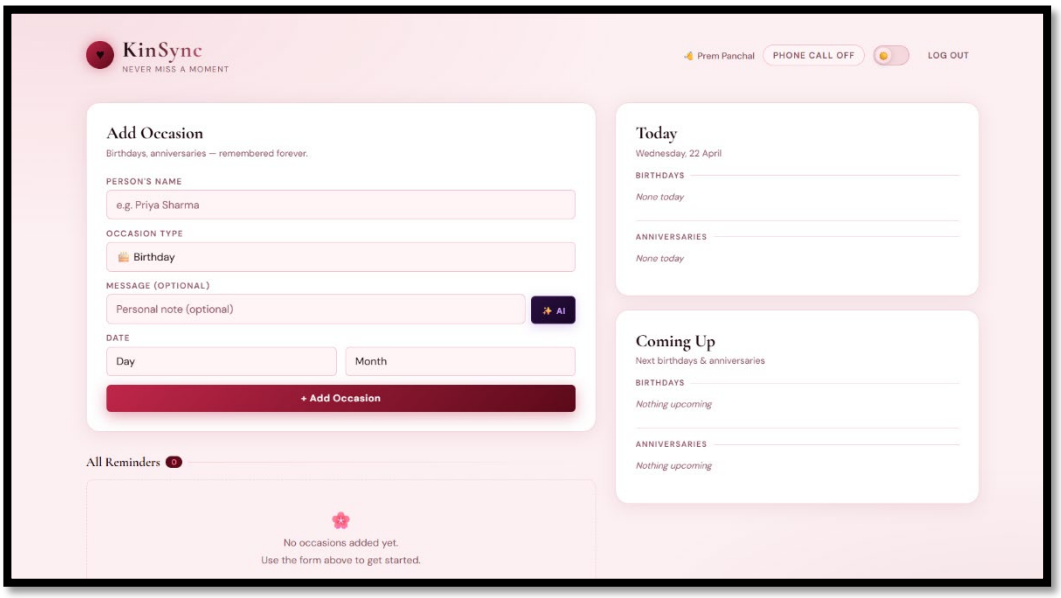


Figure 12: Dashboard (Light Theme)

➢**Database Design (MongoDB)**

users collection:

Field	Type	Notes
_id	ObjectId	Primary key
name	String	Required
email	String	Unique, lowercase
password	String	bcrypt hashed

reminders collection:

Field	Type	Notes
_id	ObjectId	Primary key
userId	ObjectId	Ref: users (FK)
personName	String	Required
occasionType	String	birthday / anniversary
remindAt	String	YYYY-MM-DD
phoneNumber	String	Optional, for call
reminderMsg	String	Optional, AI-generated
lastSentYear	Number	Prevents duplicate calls

Webpage link : KINSYNC

Conclusion

KinSync bridges the gap between technology and human connection. By combining a React frontend, Node.js backend, MongoDB database, and AI services (Gemini + Bland AI), it delivers a fully automated, personalised reminder system that requires zero ongoing effort from the user. Future scope includes WhatsApp/email notifications, Google Contacts import, and a calendar view.